

POCKETSMART AI

Your Smart Budget & Recommendation Assistant

Document	07 – Project Documentation
Project	PocketSmart AI
Prepared By	Ratchana M
Phase	Phase 7 – Documentation

1. Project Title

PocketSmart AI – Your Smart Budget & Recommendation Assistant

2. Abstract

PocketSmart AI is a web-based application that helps users plan budgets and obtain recommendations for home interiors, parties, and jewelry purchases. Users provide a budget and preferences through dedicated planner modules. The system can use Gemini AI when configured and provides a rule-based fallback when the AI service is unavailable. It also supports shopping search links and recommendation history.

3. Introduction

Budget planning can be difficult when users need to organize several expense categories within a fixed amount. PocketSmart AI combines planning, recommendation and history features in one web application.

4. Problem Statement

Users may find it difficult to divide a fixed budget across categories, identify suitable items, compare requirements, and keep previous planning information organized. Searching across multiple platforms can also require additional effort.

5. Objectives

- Provide simple budget-based planning.
- Generate recommendations from user inputs and preferences.
- Support Home, Party and Jewelry planning.
- Store recommendations for later viewing.
- Provide shopping search links.
- Maintain a simple modular web application.

6. Existing System

In a manual approach, users generally plan budgets, divide expenses, search for products, and record decisions separately. This can make planning less organized and more time-consuming.

7. Proposed System

PocketSmart AI provides a unified application for budget planning and recommendations. Users select a planner, enter budget and preferences, receive a recommendation, and can access shopping search links and saved history.

8. Technologies Used

Backend: Python, FastAPI, SQLAlchemy
Frontend: HTML, CSS, JavaScript, Jinja templates
Database: SQLite
AI: Google Gemini API (optional configuration)
Security: Session authentication and PBKDF2-SHA256 password hashing
Version Control: Git and GitHub

9. System Requirements

Software: Windows, Python, VS Code, web browser and required Python packages.
Hardware: Computer capable of running Python and a modern browser.
Optional: Gemini API key through environment configuration.

10. Project Modules

Module	Description
Authentication	Registration, login, logout and session-based access.
Home Planner	Budget-based home interior and decor planning.
Party Planner	Event planning using budget and preferences.
Jewelry Planner	Jewelry purchase planning using budget and preferences.
Recommendation Service	Gemini recommendation or rule-based fallback.
Shopping Search	Search links for relevant products.
History	Stores and displays previous recommendations.
Database	Stores user and recommendation information.

11. System Design

The application follows a modular web architecture. The browser frontend communicates with FastAPI routes. Routes use schemas, services and database components. The recommendation service can communicate with Gemini when configured and otherwise uses fallback logic.

Basic flow: User → Register/Login → Dashboard → Select Planner → Budget & Preferences → Recommendation Service → Result → Shopping Links / History

12. Database Design

SQLite is used for persistent storage. The main entities are **User** and **Recommendation**. User information supports authentication, while recommendation records support saved planning results and history.

13. Implementation

The application is implemented using FastAPI with modular route, service, schema, database, security, AI and template components. The frontend uses Jinja templates with CSS and JavaScript.

14. Testing

Functional testing covered registration, login, logout, Home Planner, Party Planner, Jewelry Planner, recommendation generation, fallback recommendation, shopping links and recommendation history. Representative budgets included ■50,000 for Home and Party planning and ■25,000 for Jewelry planning.

15. Results

The tested workflows produced the expected results for the selected inputs. Planner modules generated recommendations, shopping search links were displayed, and saved recommendations could be accessed through history.

16. Application Screenshots

The following screenshots provide visual evidence of the working PocketSmart AI application.

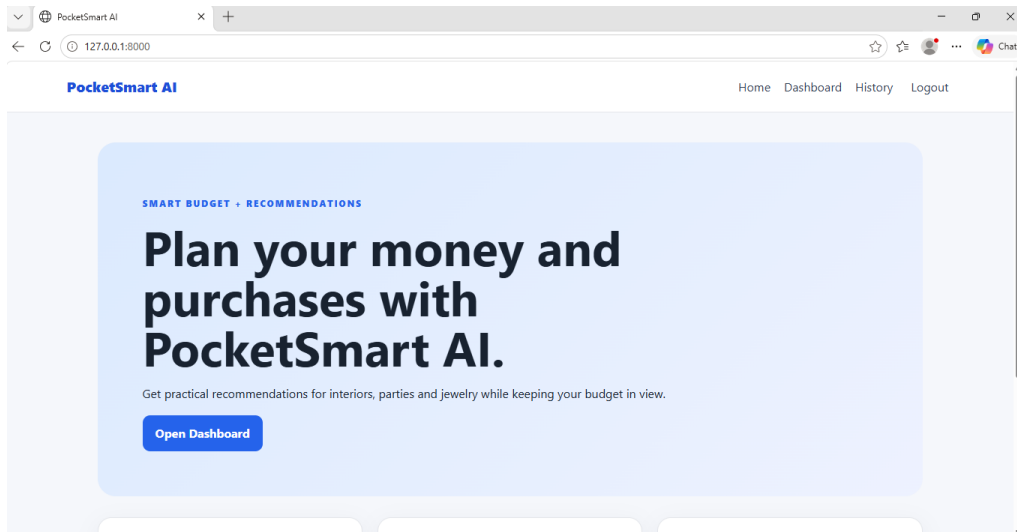


Figure 1. PocketSmart AI Home Page

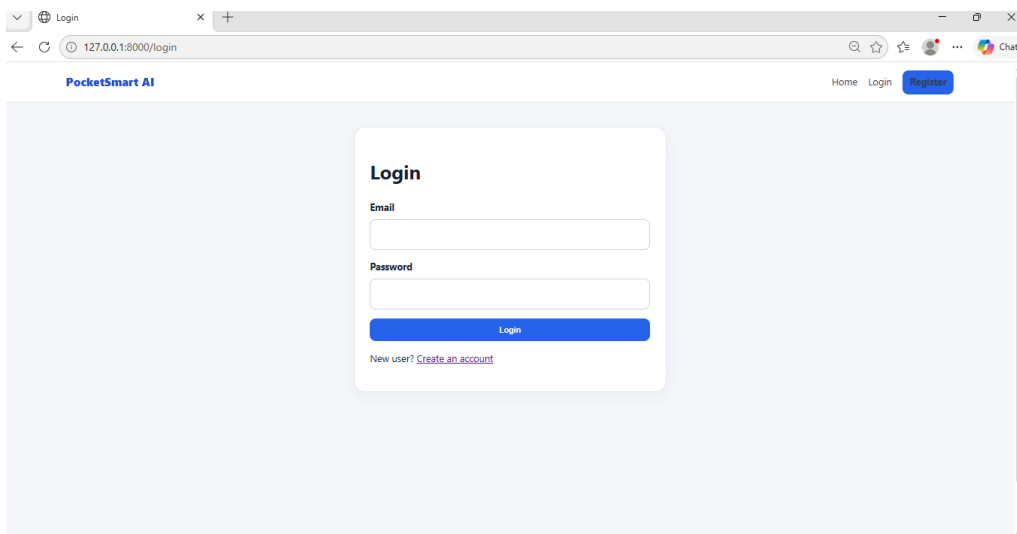


Figure 2. Login Page

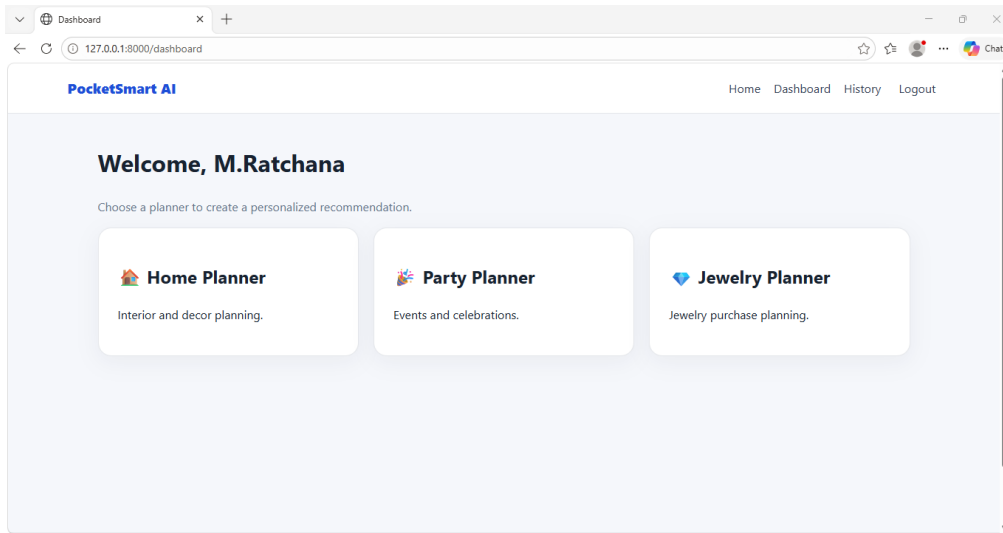


Figure 3. Dashboard with Home, Party and Jewelry Planners

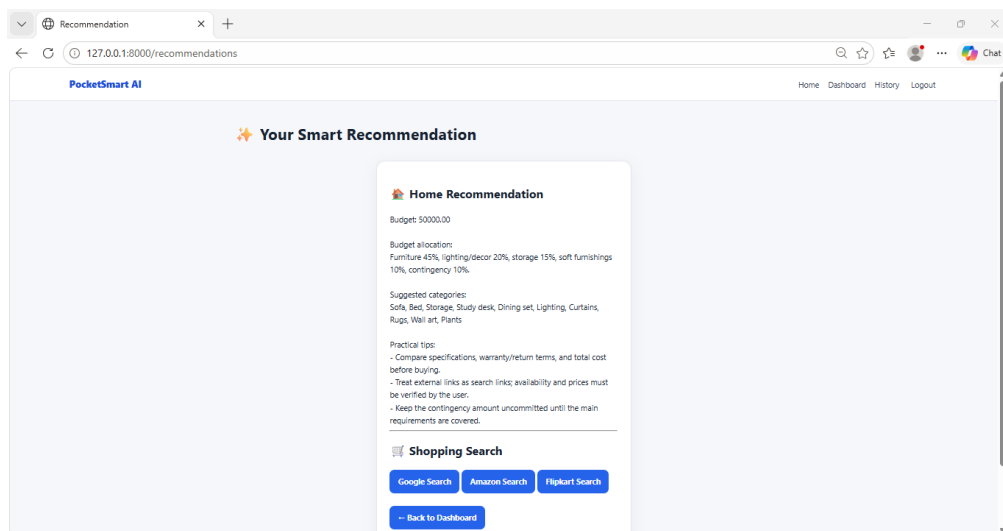


Figure 4. Recommendation Result with Shopping Search Links

17. Conclusion

PocketSmart AI provides a structured approach to budget planning and recommendations through Home, Party and Jewelry planners. The application combines authentication, recommendation generation, shopping search links and history management in a modular FastAPI-based web application.

18. Future Enhancements

- Add more planning domains and recommendation categories.
- Improve recommendation personalization.
- Add richer product comparison features.
- Add advanced AI-assisted planning capabilities.
- Improve reporting and visualization of saved history.

19. Installation & Setup

1. Clone or download the project repository.
2. Create and activate a Python virtual environment.
3. Install packages from **requirements.txt**.
4. Configure local settings using **.env.example**.
5. Start the FastAPI application using Uvicorn.
6. Open the application in a web browser and register/login.

20. Project Repository

GitHub repository: [ratchana112007/POCKETSMART_AI](https://github.com/ratchana112007/POCKETSMART_AI)

Security Note: Real passwords, API keys and other secrets should not be included in documentation or source control. Use the environment example for configuration guidance.